On-Prem to Azure Ingestion with Data Factory and SHIR

Nov 1, 2026

What you're building

A Data Factory pipeline picks up every `*.csv` file in `C:\outbox` on your own Windows machine, uploads it to the `landing` container in the data lake, and then moves the original into `C:\outbox\archive`. The machine is reached through a **self-hosted integration runtime (SHIR)**: a Windows service that only makes outbound HTTPS connections to Azure, so no inbound ports are opened.

```
Your laptop (Windows)                        Azure
+---------------------------+               +--------------------------------------+
| C:\outbox\*.csv           |               | Data Factory (managed identity)      |
| C:\outbox\archive\        |<--- SHIR ----->|  +- ls_outbox   (FileServer via SHIR)|
| SHIR (Integration Runtime)|  outbound 443  |  +- ls_datalake (ADLS, managed ID)   |
+---------------------------+               |  +- ls_keyvault (password for outbox)|
                                            |  +- pipeline: list > copy > archive  |
                                            | Storage: landing/   Key Vault        |
                                            +--------------------------------------+
```

Data Factory sends the SHIR instructions ("list `C:\outbox`", "read this file"); the SHIR runs them locally and streams the data up.

The pipeline: `pl_outbox_to_landing`

```
ListOutbox (Get Metadata: childItems of C:\outbox)
   |
   v
OnlyCsv (Filter: type = File AND name ends with .csv)
   |
   v
EachFile (ForEach, up to 4 files in parallel)
   |
   +--> CopyToLanding : C:\outbox\<file>  -->  landing/<file>
            |
            | only on Succeeded
            v
        ArchiveSource : C:\outbox\<file>  -->  C:\outbox\archive\<file>
                        then delete C:\outbox\<file>   (= a move)
```

Why the design looks like this

- **List the files, then handle each one.** A single wildcard copy followed by a wildcard delete could delete a file that arrived between the two steps without ever uploading it. This way, only files that were actually copied get archived.
- **The archive step only runs after a successful upload.** If the upload fails, the file stays in `C:\outbox` and is retried on the next run.
- **No secrets in Terraform.** The password lives only in Key Vault. ADF fetches it at run time using its managed identity.

Where it lives in the repo

The assignment asks for Bicep; this project uses Terraform, so the factory is a Terraform module. The `landing` and `archive` containers already exist in `modules/storage`.

File	What it holds
`infra/modules/datafactory/main.tf`	Factory (system-assigned identity), 2 role assignments, SHIR, 3 linked services, 4 datasets, the pipeline
`infra/modules/datafactory/variables.tf`	Inputs, incl. `outbox_path`, `outbox_user`, `outbox_password_secret_name`
`infra/modules/datafactory/outputs.tf`	`name`, `id`, `principal_id`, `shir_name`, `shir_auth_key` (sensitive)
`infra/modules/storage/outputs.tf`	Adds `dfs_endpoint` for the lake linked service
`infra/main.tf`	`module "adf"` wiring
`infra/variables.tf`	`outbox_path` (default `C:\outbox`), `outbox_user`
`infra/env/dev.tfvars`	`outbox_user = "VILSHUB-NET\\adfreader"`

What the module creates:

Resource	Name	Purpose
Data factory	`adf-ingest-<env>-<suffix>`	Runs the pipeline; has a system-assigned managed identity
Role assignment	Storage Blob Data Contributor on the storage account	Factory can write to `landing`
Role assignment	Key Vault Secrets User on the vault	Factory can read the outbox password
Integration runtime	`shir-onprem`	The self-hosted runtime your laptop registers to
Linked service	`ls_keyvault`	Points at the Key Vault
Linked service	`ls_datalake`	ADLS Gen2, signs in with the managed identity
Linked service	`ls_outbox`	FileServer at `C:\outbox`, connects via `shir-onprem`, password from Key Vault
Datasets	`ds_outbox_folder`, `ds_outbox_file`, `ds_outbox_archive`, `ds_landing_file`	Binary datasets (files copied byte for byte); the file ones take a `fileName` parameter
Pipeline	`pl_outbox_to_landing`	The flow above

Step 0: Prerequisites (once)

1. Register the Data Factory provider. Same reason as Key Vault in the OIDC guide: the pipeline identity cannot register providers.

- Portal: Subscriptions, your subscription, Settings, Resource providers, filter `Microsoft.DataFactory`, select it, Register.
- CLI:

```
az provider register -n Microsoft.DataFactory --wait
```

2. Let the pipeline identity create role assignments. The module grants the factory access to storage and Key Vault. The pipeline identity is only Contributor on `rg-ingest-dev`, and Contributor cannot create role assignments, so the apply fails with `AuthorizationFailed ... roleAssignments/write`.

The fix is **Role Based Access Control Administrator** on `rg-ingest-dev`, with a **condition** that limits which roles it may hand out. The condition must allow exactly the roles the module assigns:

Role Terraform assigns	Role definition ID
Storage Blob Data Contributor	`ba92f5b4-2d11-453d-a403-e96b0029c9fe`
Key Vault Secrets User	`4633458b-17de-408a-b874-0445c86b69e6`

Portal (this is the route that worked in the worked example):

Part A: remove a wrong assignment first, if one exists. You cannot fix a bad condition by adding a second assignment.

1. Resource groups, `rg-ingest-dev`, **Access control (IAM)**, **Role assignments** tab.
2. Search for your app (or filter **Type**, **Apps**).
3. Tick the **Role Based Access Control Administrator** row whose Scope says *This resource*, click **Delete**, **Yes**.

Part B: add it with the right condition.

1. Still on **Access control (IAM)**, click **+ Add**, **Add role assignment**.
2. **Role** tab: click the **Privileged administrator roles** sub-tab, select **Role Based Access Control Administrator**, **Next**.
3. **Members** tab: **Assign access to** = *User, group, or service principal*. Click **+ Select members**, paste the client ID `4b0e464d-9fa9-4540-9095-853f79bdf0b7` into the search (searching by ID, not name, means you cannot pick the wrong identity), click the app, **Select**, **Next**.
4. **Conditions** tab, under *What user can do*: choose **Allow user to only assign selected roles to selected principals (fewer privileges)**, then click **+ Select roles and principals**.
5. A template picker opens with four cards. Click **Configure** on the right one:

Card	Use it?	Why
Constrain roles	**Yes** (simplest)	Limits which roles can be handed out, to anyone in the resource group
Constrain roles and principal types	Yes (a bit tighter)	Same two roles, and set principal type to **Service principals**: the Data Factory's managed identity is a service principal, so users and groups can never receive these roles
Constrain roles and principals	**No**	Locks the target to specific principal IDs. In the worked example the pipeline app itself was picked, so assigning to the Data Factory's identity was rejected. Even with the right ID, it breaks whenever the factory is recreated (new identity, new ID)
Allow all except specific roles	No	Allows everything not listed; too broad

6. Under **Roles**, click **+ Add roles**, search and tick **Storage Blob Data Contributor** and **Key Vault Secrets User**, **Select**. Make sure only these two are listed (remove anything else). **Save**.
7. Back on the Conditions tab the summary reads *Allow user to assign: Storage Blob Data Contributor, Key Vault Secrets User*. **Next**.
8. **Assignment type** tab (if shown): **Active**, **Permanent**, **Next**.
9. **Review + assign**, **Review + assign**.

Part C: check it.

1. **Role assignments** tab, find the new row, in the **Condition** column click **View/Edit**.
2. You should see Action = *Create or update role assignments* and *Delete role assignments*, Role = the two roles above, and **no principal** constraint (or only *Service principals* if you used the second card).
3. Wait about 5 minutes, then GitHub, **Actions**, the failed run, **Re-run failed jobs**. The apply finishes with `Apply complete! Resources: 2 added, 1 changed, 0 destroyed.`

What each setting controls:

Wizard setting	Controls	Value here
Role	What the pipeline may do: create/delete role assignments	Role Based Access Control Administrator
Scope (where you clicked + Add)	Where it may do that	Only inside `rg-ingest-dev`
Condition: roles	**Which** roles it may hand out	Only Storage Blob Data Contributor and Key Vault Secrets User
Condition: principals	**Who** may receive them	Anyone (or service principals only)

Net effect: the pipeline can give two specific roles inside one resource group and nothing else. It can never make itself or anyone else Owner.

CLI (PowerShell in Cloud Shell; deletes any existing assignment and recreates it with the right condition):

```
$scope = "/subscriptions/147103a9-1f23-4c65-ae33-96dac6063586/resourceGroups/rg-ingest-dev"
$cond = @'
((!(ActionMatches{'Microsoft.Authorization/roleAssignments/write'})) OR (@Request[Microsoft.Authorization/roleAssignments:RoleDefinitionId] ForAnyOfAnyValues:GuidEquals {ba92f5b4-2d11-453d-a403-e96b0029c9fe, 4633458b-17de-408a-b874-0445c86b69e6})) AND ((!(ActionMatches{'Microsoft.Authorization/roleAssignments/delete'})) OR (@Resource[Microsoft.Authorization/roleAssignments:RoleDefinitionId] ForAnyOfAnyValues:GuidEquals {ba92f5b4-2d11-453d-a403-e96b0029c9fe, 4633458b-17de-408a-b874-0445c86b69e6}))
'@
az role assignment delete --assignee 4b0e464d-9fa9-4540-9095-853f79bdf0b7 --role "Role Based Access Control Administrator" --scope $scope
az role assignment create --assignee 4b0e464d-9fa9-4540-9095-853f79bdf0b7 --role "Role Based Access Control Administrator" --scope $scope --condition $cond --condition-version "2.0"
```

Check it (the Condition column must list both IDs above):

```
az role assignment list --assignee 4b0e464d-9fa9-4540-9095-853f79bdf0b7 --all --query "[].{role:roleDefinitionName, scope:scope, condition:condition}" -o table
```

Wait about 5 minutes for the new permission to reach the pipeline's token, then re-run the failed job (Actions, the run, **Re-run failed jobs**). Do the same on `rg-ingest-prod` before the first prod deploy.

Why constrained: without a condition the pipeline could grant any role, including Owner, inside the resource group. With it, the pipeline can only ever hand out these two. If you later add Terraform that assigns another role, add that role's ID to the condition. Look up any ID with:

```
az role definition list --name "<role name>" --query "[].name" -o tsv
```

3. Create a dedicated Windows account and the folders. Do not use your personal login password. PowerShell as Administrator:

```
net user adfreader "<StrongPassword>" /add
mkdir C:\outbox\archive
icacls C:\outbox /grant "adfreader:(OI)(CI)M"
```

Step 1: Deploy the Terraform

Push to the `dev` branch. The `infra` workflow runs `terraform apply` and creates everything in the table above. Check in the portal that `adf-ingest-dev-...` exists in `rg-ingest-dev`.

Step 2: Put the password in Key Vault (manual, once)

Terraform never holds the password (it would end up in the state file). The vault uses RBAC, so first give yourself **Key Vault Secrets Officer** on it (vault, Access control (IAM), Add role assignment).

- Portal: the vault, **Objects**, **Secrets**, **+ Generate/Import**, Name `outbox-password`, Secret value = the `adfreader` password, **Create**.
- CLI:

```
az keyvault secret set --vault-name <kv-ingest-dev-...> -n outbox-password --value '<adfreade
r password>'
```

Step 3: Install and register the SHIR

1. Get the authentication key, either way:
   - Portal: Data Factory, **Launch Studio**, **Manage** (toolbox icon), **Integration runtimes**, `shir-onpre m`, copy **Key1**.
   - Terraform: `terraform output -raw shir_auth_key`.
2. Download the installer: search "Microsoft Integration Runtime download" or open `https://www.micro soft.com/download/details.aspx?id=39717`, pick the latest `IntegrationRuntime_5.x.msi`.
3. Run it on **Windows** (not inside WSL), keep the defaults, **Finish**.
4. Configuration Manager opens: paste the key, **Register**, **Finish**.
5. In ADF Studio, Manage, Integration runtimes: `shir-onprem` shows **Running**.
6. Newer SHIR versions block local paths like `C:\outbox` by default. If a run fails with a "local folder path" / "not allowed" error, PowerShell as Administrator:

```
cd "C:\Program Files\Microsoft Integration Runtime\5.0\Shared"
.\dmgcmd.exe -DisableLocalFolderPathValidation
```

Step 4: Test it

1. Copy `samples/orders.csv` from the repo into `C:\outbox\`.

2. ADF Studio, **Author**, Pipelines, `pl_outbox_to_landing`, **Debug** (or **Add trigger**, **Trigger now**; see below).
3. Check:
   - Storage account, `landing` container: `orders.csv` is there.
   - `C:\outbox\orders.csv` is gone; `C:\outbox\archive\orders.csv` exists.
   - **Monitor** tab: the run is green and every activity succeeded.

Running it from Studio. In the pipeline toolbar, **Add trigger** opens a small menu with two items:

- **Trigger now**: runs the published pipeline once. Use this for a manual run.
- **New/Edit**: opens the *New trigger* form (Name, Type = Schedule, Start date, Recurrence...). That creates a **permanent schedule**. Cancel it if you only wanted a test run, and do not create schedules in the UI at all: Terraform does not know about them, so they are not in your code or in prod. Add schedules as `azurerm_data_factory_trigger_schedule` in the module instead.

Debug (same toolbar) also runs the pipeline immediately and is better for a first test: the **Output** tab at the bottom lists every activity with its status, duration and integration runtime, and you can open each one's input, output and error.

What a successful run looks like (worked example, run on Nov 2, 2026):

Activity	Type	Status	Duration	Integration runtime
ListOutbox	Get Metadata	Succeeded	16s	`shir-onprem`
OnlyCsv	Filter	Succeeded	< 1s	(runs in Azure)
EachFile	ForEach	Succeeded	44s	(runs in Azure)
CopyToLanding	Copy data	Succeeded	19s	`shir-onprem`
ArchiveSource	Copy data	Succeeded	21s	`shir-onprem`

The three activities that touch `C:\outbox` ran on `shir-onprem`, which proves the SHIR, the Key Vault password and both role assignments all work. One CopyToLanding/ArchiveSource pair means one CSV was processed; with three CSVs you would see three pairs.

Further tests worth running:

- Put two or three `.csv` files and one `.txt` file in `C:\outbox`. Every CSV gets its own pair; the `.txt` is left alone.
- Run with an empty `C:\outbox`. The run succeeds and EachFile does nothing.

Understanding what you built

Where the pipeline lives: Terraform defines it, ADF runs it

The pipeline is **defined in Terraform** and **deployed to, stored in and run by Azure Data Factory**. Think blueprint and building: Terraform is the blueprint in your repo; `terraform apply` sends it to Azure; ADF holds the real pipeline and executes it when you click Debug or a trigger fires.

What you see in ADF Studio	Where it is defined
The factory `adf-ingest-dev-...`	`infra/modules/datafactory/main.tf`, `azurerm_data_factory.adf`; wired into the project by `module "adf"` in `infra/main.tf`
Integration runtime `shir-onprem`	`azurerm_data_factory_integration_runtime_self_hosted.shir`
Linked services `ls_keyvault`, `ls_datalake`, `ls_outbox` (Studio: **Manage**, **Linked services**)	`azurerm_data_factory_linked_service_key_vault.kv`, `..._data_lake_storage_gen2.lake`, `azurerm_data_factory_linked_custom_service.outbox`
Datasets `ds_outbox_folder`, `ds_outbox_file`, `ds_outbox_archive`, `ds_landing_file`	`azurerm_data_factory_custom_dataset.*`
Pipeline `pl_outbox_to_landing` with ListOutbox, OnlyCsv, EachFile, CopyToLanding, ArchiveSource	`azurerm_data_factory_pipeline.ingest`, inside `activities_json = jsonencode([...])`

Rule: **change it in Terraform, run and monitor it in Studio.** The factory is not connected to Git, so edits made in Studio go straight to the live factory, and the next `terraform apply` sees the difference and overwrites them. Edit `main.tf`, push to `dev`, let the CI/CD pipeline deploy it.

Seeing the code inside ADF

Every pipeline, dataset and linked service is stored in ADF as JSON. To view it:

1. **Pipeline:** ADF Studio, **Author** (pencil icon), **Pipelines**, `pl_outbox_to_landing`, then the `{ }` (Code) icon at the top right of the canvas, next to the Properties icon.
2. **Datasets: Author**, **Datasets**, pick one, same `{ }` icon.
3. **Linked services: Manage** (toolbox icon), **Linked services**, hover a row, click its `{ }` icon. Same for **Integration runtimes**.

In `ls_outbox` you will see the password is a Key Vault reference (`"type": "AzureKeyVaultSecret"`, `"secretName": "outbox-password"`), not the password itself.

The JSON is exactly what `jsonencode(...)` in Terraform produced: same names, same structure (`name`, `type`, `dependsOn`, `typeProperties`). ADF adds a few default fields of its own (`policy`, `userProperties`, ...), which is why every Terraform plan shows `1 to change` on the pipeline. It is harmless.

The `@...` syntax: the ADF expression language

The values starting with `@` look a little like PowerShell but are not. They are the **Azure Data Factory expression language**, a small built-in language for dynamic values. Synapse pipelines use the very same language.

Expression in this pipeline	Meaning
`@activity('ListOutbox').output.childItems`	The output of ListOutbox: the list of items in `C:\outbox`
`@item()`	The current element inside a Filter or ForEach, i.e. one file
`@item().name` / `@item().type`	That file's name (`orders.csv`) and type (`File` or `Folder`)
`@and(equals(item().type,'File'), endswith(toLower(item().name),'.csv'))`	True only for files whose name ends in `.csv`, any case
`@activity('OnlyCsv').output.value`	The filtered list that ForEach loops over
`@dataset().fileName`	Inside a dataset: the `fileName` parameter the activity passed in

How to read it:

- `@` at the start means "evaluate this", not literal text.
- Functions are prefix style: `equals(a, b)`, never `a == b`. Common ones: `and`, `or`, `not`, `equals`, `endswith`, `startswith`, `toLower`, `concat`, `utcNow`, `formatDateTime`.
- In the JSON an expression is wrapped as `{ "value": "@...", "type": "Expression" }` so ADF knows to evaluate it.
- In Studio, any field with **Add dynamic content** opens the expression builder, which lists every function and every earlier activity's output.

Two different "pipelines"

This project has two things called a pipeline. They do different jobs:

	ADF pipeline	GitHub Actions pipeline
Name	`pl_outbox_to_landing`	the `infra` workflow (`.github/workflows/infra.yml` + `deploy.yml`)
Kind	**Data pipeline** (data integration / ETL)	**CI/CD pipeline** (deployment)
Moves	**Data**: CSVs from `C:\outbox` to `landing/`	**Infrastructure**: runs Terraform to create Azure resources
Runs on	Azure Data Factory, plus the SHIR on your laptop	GitHub-hosted runners
Defined in	`infra/modules/datafactory/main.tf`	the two workflow YAML files
Started by	Debug / Trigger now (a schedule later)	push to `dev` or `prod`, or a pull request
Signs in to Azure with	the factory's **managed identity**	**OIDC** with the app registration (see the OIDC setup guide)

How they connect: **the GitHub Actions pipeline deploys the ADF pipeline.** You push a Terraform change, the CI/CD pipeline runs `terraform apply`, which creates or updates `pl_outbox_to_landing`; from then on ADF runs it to move files.

Data Factory vs Synapse Analytics

Data Factory moves and orchestrates data. Synapse moves, orchestrates *and* processes it.

	Data Factory	Synapse Analytics
Main job	Move and orchestrate data	Move and orchestrate data **plus** query, transform and warehouse it
Pipelines	Yes	Yes (**same engine as ADF**)
SQL over files in the lake	No	Yes: **serverless SQL** (`SELECT ... FROM OPENROWSET (...)` straight on CSV/Parquet in storage)
Data warehouse	No	Yes: **dedicated SQL pool** (MPP warehouse)
Spark (Python/Scala notebooks)	No (only by calling Databricks/HDInsight)	Yes: **Spark pools**
Packaging	Standalone factory resource	A **workspace** holding pipelines, SQL, Spark and notebooks together
Typical cost	Per activity run, cheap	Serverless SQL per query (cheap); dedicated pools and Spark cost more

Does Synapse orchestrate? Yes. **Synapse pipelines** are the same orchestration engine as Data Factory: the same activities (Copy, Get Metadata, Filter, ForEach, If, Wait, Web...), the same `@` expression language, the same triggers (schedule, tumbling window, storage event), self-hosted integration runtimes, monitoring and reruns. `pl_outbox_to_landing` would move across with almost no changes. On top of that, a Synapse pipeline can drive Synapse's own compute: run a Spark notebook or a SQL script, then carry on. Minor differences: some ADF features reach Synapse later or not at all (certain connectors, the SSIS integration runtime).

When you need Synapse:

- To **query lake files with SQL** without loading them anywhere, e.g. total orders per day across every CSV in `curated/`.
- For **heavy transformations** (joins, aggregations, deduplication over millions of rows) that a Copy activity cannot do.
- To **serve a warehouse** (star schemas, fast Power BI dashboards).
- To keep **movement and processing in one workspace** instead of ADF + Databricks + a SQL database.

When you do not: when the job is moving files from A to B, or transformations are light. This step (`C:\outbox` to `landing/` to archive) is pure movement, so ADF is the right tool.

Rule of thumb: orchestration only (copy, move, call other services) means **ADF**; orchestration plus SQL/Spark transformation in one place means **Synapse** (or Fabric).

Where it fits in this project: the lake already has `landing`, `raw`, `curated`, `quarantine` and `archive`, and `pipelines/validate.py` checks data quality. Synapse (or an equivalent) would come in at the **raw to curated** stage if the course has you transform, aggregate or serve the data with SQL.

Microsoft Fabric: Microsoft's newer analytics platform. It covers what Synapse does (pipelines, Spark, SQL, warehouse) plus Power BI, as one SaaS product. Synapse still works and is widely used, but Fabric is where Microsoft's investment is going; prefer it if a later module offers the choice.

Troubleshooting

Error	Fix
`MissingSubscriptionRegistration ... Microsoft.DataFactory`	Step 0.1
`AuthorizationFailed: ... does not have authorization to perform action 'Microsoft.Authorization/roleAssignments/write'` during apply	The identity has no Role Based Access Control Administrator on the resource group (or it went to the wrong identity or scope). Run the check command in Step 0.2; add the role
`AuthorizationFailed: ... has an authorization with ABAC condition that is not fulfilled`	The role exists, but its condition blocks this assignment. Open the condition (**View/Edit**) and check two things: both role IDs from Step 0.2 are listed, and there is **no** `PrincipalId` constraint (the *Constrain roles and principals* card). Recreate it with the *Constrain roles* card
Same 403 right after granting the role	Not propagated yet ("If access was recently granted, please refresh your credentials"). Wait about 5 minutes and re-run
Plan shows `1 to change` on `pl_outbox_to_landing` every run	Azure stores the pipeline JSON in a slightly different form than Terraform sends it. Harmless
Linked service test: `Access denied` / `Logon failure`	`outbox_user` must be `HOSTNAME\adfreader`; the Key Vault secret must match the password; the account needs rights on `C:\outbox`
`Forbidden` reading the secret	Factory identity lacks Key Vault Secrets User (Terraform adds it; allow a few minutes to propagate)
`403` writing to `landing`	Storage Blob Data Contributor for the factory has not propagated yet; wait and re-run
Local folder path not allowed	Step 3.6
SHIR shows **Unavailable**	Laptop off or asleep, or the service stopped; open Configuration Manager

Things to know

- **Same file name overwrites.** A second `orders.csv` replaces the first one in `landing`. To keep both, give the landing dataset a date folder, e.g. `folderPath = "@formatDateTime(utcNow(),'yyyy/MM/dd')"`.

- **No schedule yet.** The pipeline runs only when triggered. Add `azurerm_data_factory_trigger_schedule` to run it every N minutes.
- **One SHIR, one factory.** A SHIR install registers to a single data factory. The laptop serves `dev`; prod needs its own SHIR machine.